

# Remove span's `initializer_list` constructor for C++26

Document #: P4144R1  
Date: 2026-03-26  
Project: Programming Language C++  
LEWG  
Reply-to: Mark Hoemmen  
<[mark.hoemmen@gmail.com](mailto:mark.hoemmen@gmail.com)>  
Tomasz Kamiński  
<[tomaszkam@gmail.com](mailto:tomaszkam@gmail.com)>  
Tim Song  
<[t.canens.cpp@gmail.com](mailto:t.canens.cpp@gmail.com)>  
Jonathan Wakely  
<[cxx@kayari.org](mailto:cxx@kayari.org)>

## Contents

- 1 Revision history
- 2 Problem: “Silent” change in span’s behavior
  - 2.1 Summary
  - 2.2 Cause: New `initializer_list` constructor
- 3 It’s not “silent,” but implementations diverge
- 4 P2447 introduced Standard Library and wording bugs, later fixed
- 5 Proposed fix: Remove `initializer_list` constructor
- 6 Alternatives
- 7 Dissenting opinion
- 8 Proposed wording
  - 8.1 Remove `__cpp_lib_span_initializer_list` macro
  - 8.2 Remove `initializer_list` include from `[span.syn]`
  - 8.3 Remove `initializer_list` constructor from span’s class declaration
  - 8.4 Remove `initializer_list` constructor from span’s description
  - 8.5 Remove the entire `[diff.cpp23.containers]` section
- 9 Appendix A: Original (pre-LEWG-review) proposed fix

- [9.1 Ansatz: Constrain the initializer list constructor](#)
- [9.2 Fix initializer list construction from actual bool values](#)
- [9.3 Fix initializer list construction from any non-pointer type convertible to bool?](#)

## § 1 Revision history

R0 with title “Breaking change in `std::span`’s new `initializer_list` constructor” was reviewed by LEWG on 2026-03-25. That version of the paper proposed a fix just for `span<const bool>`. It was submitted into the `isocpp.org` paper system as R0 immediately before LEWG review; it had not reached an official mailing. On 2026-03-25, LEWG polled 16/14/2/0/0 instead to remove the `initializer_list` constructor from `span` entirely for C++26, and asked the paper to be revised. Tomasz Kamiński, Tim Song, and Jonathan Wakely joined the author list.

R1 shows the post-LEWG-review design. Please see Appendix A for the originally proposed fix.

## § 2 Problem: “Silent” change in `span`’s behavior

### § 2.1 Summary

This paper expresses [LWG4520](#) and proposes fixing it by reverting adoption of [P2447R6](#).

LWG 4520 came from [an issue](#) filed on NVIDIA’s Standard Library’s (`libc++`) implementation of `span`. The `libc++` authors found that they had correctly implemented the specification and that the specification change itself caused the issue.

We’ve written a demo (also with partial implementation of proposed fix) [here](#). Here is a short illustration of the change in behavior between C++23 and C++26.

```
#include <span>

int main() {
    bool data[4] = {true, false, true, false};
    bool* ptr = data;
    size_t size = 4;

    std::span<bool> span_nc{ptr, size};
    // OK in both C++23 and C++26
    assert(span_nc.size() == size);

    std::span<const bool> bad_span{ptr, size};
    // OK in C++23, but FAILS in C++26
    assert(bad_span.size() == size);
}
```

```
return 0;
}
```

## § 2.2 Cause: New `initializer_list` constructor

In C++23, constructor `span(element_type*, size_t)` is called.

WG21 adopted [P2447R6](#) for C++26 at the March 2024 Tokyo meeting. This added a new `span(initializer_list<value_type>)` constructor. As a result, in the above example, the new constructor is selected instead. It follows the conversion sequence from `bool*` to `bool` and `size_t` to `bool`.

P2447 authors understood that adoption of their proposal would be a breaking change. The paper adds to Annex C some examples of code that this breaks. This includes “silent” semantic changes to code, such as the following.

```
void *a[10];
// x is 2; previously 0
int x = span<void* const>{a, 0}.size();
any b[10];
// y is 2; previously 10
int y = span<const any>{b, b+10}.size();
```

The use case in [LWG 4520](#) is most like the `span<const any>` break above, in that both `ElementType*` and `size_t` are convertible to the span’s `value_type`.

However, `span<const any>` is a less common use case. Creating a `span<const bool>` is likely more common, especially in generic code. That means LEWG might not have realized the significance of the change.

Note that `span<bool>` does not have this issue. Using `{}` with `mdspan` (the source of design inspiration for `span`) works as expected.

## § 3 It’s not “silent,” but implementations diverge

The example is actually ill-formed code; it’s a narrowing conversion from pointer to `bool`. GCC trunk compiles this but emits narrowing warnings. Clang stops with an error.

Per [\[intro.compliance.general\] 2.3](#), both implementations are conforming, as they emit a “diagnostic.” GCC implements a conforming “extension” per [\[intro.compliance.general\] 11](#), in that it attempts to give meaning (by compiling) to invalid code.

In general, GCC treats narrowing with constant expressions as an error, but for non-constant expressions only issues a warning, on the basis that the existing, working code doing it is probably correct. GCC made this choice consciously, as a way to promote adoption of C++11. For example, GCC makes the following an error,

```
char str[] = { 300, 400, '\0' };
```

but only issues a warning for the following.

```
char str[] = { tolower(ch1), tolower(ch2), '\0' };
```

This is a narrowing conversion and therefore ill-formed because `tolower` returns `int` not `char`. However, in practice, it's never lossy; the `tolower` value always fits in `char`. Rejecting code like this because of the C++11 narrowing rules would have hindered adoption of C++11 enormously.

## § 4 P2447 introduced Standard Library and wording bugs, later fixed

Adoption of P2447 led to bugs in both Standard Library implementations and the Standard wording that later had to be fixed. We already mentioned the `libc++` bug above. The `libstdc++` [Bug 120997](#) led to filing of [LWG 4293](#). The Standard was specified to use curly braces for the return values of some `span` member functions, such as `submdspan`. Adoption of [LWG 4293](#)'s Proposed Fix corrected that.

## § 5 Proposed fix: Remove `initializer_list` constructor

On 2026-03-25, LEWG polled 16/14/2/0/0 to remove the `initializer_list` constructor from `span` entirely for C++26. LEWG asked this paper to be revised to include that solution. Appendix A shows the originally proposed solution.

As far as we know, this should only affect wording of the `span` specification itself. Adoption of the proposed fix for [LWG 4293](#) changed the wording to stop using the `initializer_list` constructor where doing so was not intended.

## § 6 Alternatives

1. File a GCC bug to make GCC emit an error here.
2. Try to constrain `span<T>`'s constructor generically (for all `T`) so that it refuses to convert a pointer to `value_type`.

Regarding Option (1), GCC has made a principled choice here to emit warnings in some cases. Discussion during LEWG review suggests that they might be willing to consider emitting errors for more targeted situations.

Given that LEWG has asked us to remove the `initializer_list` constructor entirely, we can always consider Option (2) for later C++ versions. The `span` class template has many constructors with intricate constraints, so this will need to be done carefully.

One suggestion is to constrain the constructor as follows.

```
template <std::same_as<value_type> InitListValueType>
constexpr
    my_span(std::initializer_list<const InitListValueType> il)
        requires (is_const_v<ElementType>);
```

This would prohibit the troublesome cases discussed above. However, it would also change the original design by forcing the type match to be exact. For example, `span<const bool>{true, false, true}` would work, but `span<const bool>{1, 0, 1}` would fail overload resolution.

## § 7 Dissenting opinion

One of P2447's authors, Arthur O'Dwyer, contacted Tomasz Kamiński and myself to ask us to link to [this blog post](#) about successful use of the `span(initializer_list<value_type>)` constructor in Chromium.

This could be relevant for discussion of restoring the removed feature in future C++ versions.

## § 8 Proposed wording

### § 8.1 Remove `__cpp_lib_span_initializer_list` macro

[ Editor's note: In [version.syn], remove the `__cpp_lib_span_initializer_list` feature test macro definition. ]

```
#define __cpp_lib_span 202311L //
    freestanding, also in <span>

#define __cpp_lib_span_initializer_list 202311L //
    freestanding, also in <span>

#define __cpp_lib_spanstream 202106L // also in
    <iosfwd>, <spanstream>
```

### § 8.2 Remove `initializer_list` include from [span.syn]

[ Editor's note: Remove `#include <initializer_list>` from [span.syn] ("Header <span> synopsis") as shown below. ]

```
#include <initializer_list> // see [initializer.list.syn]

// mostly freestanding
namespace std {
```

### § 8.3 Remove `initializer_list` constructor from `span`'s class declaration

[ Editor's note: Remove `span(initializer_list<value_type>)` constructor from `span`'s class declaration in `[span.overview]`, as shown below. ]

```
// [span.cons], constructors, copy, and assignment
constexpr span() noexcept;
template<class It>
    constexpr explicit(extent != dynamic_extent) span(It first,
        size_type count);
template<class It, class End>
    constexpr explicit(extent != dynamic_extent) span(It first, End
        last);
template<size_t N>
    constexpr span(type_identity_t<element_type> (&arr)[N]) noexcept;
template<class T, size_t N>
    constexpr span(array<T, N>& arr) noexcept;
template<class T, size_t N>
    constexpr span(const array<T, N>& arr) noexcept;
template<class R>
    constexpr explicit(extent != dynamic_extent) span(R&& r);
```

```
constexpr explicit(extent != dynamic_extent)
    span(std::initializer_list<value_type> il);
```

```
constexpr span(const span& other) noexcept = default;
template<class OtherElementType, size_t OtherExtent>
    constexpr explicit(see below) span(const span<OtherElementType,
        OtherExtent>& s) noexcept;
```

## § 8.4 Remove `initializer_list` constructor from `span`'s description

[ Editor's note: Remove `span(initializer_list<value_type>)` constructor description from `[span.cons]` (paragraphs 21-23), as shown below. ]

19 *Effects:* Initializes `data_` with `ranges::data(r)` and `size_` with `ranges::size(r)`.

20 *Throws:* What and when `ranges::data(r)` and `ranges::size(r)` throw.

```
constexpr explicit(extent != dynamic_extent)
    span(std::initializer_list<value_type> il);
```

21 *Constraints:* `is_const_v<element_type>` is true.

22 *Hardened preconditions:* If `extent` is not equal to `dynamic_extent`, then `il.size() == extent` is true.

23 *Effects:* Initializes `data_` with `il.data()` and `size_` with `il.size()`.

```
constexpr span(const span& other) noexcept = default;
```

24 *Postconditions:* `other.size() == size()` && `other.data() == data()`.

## § 8.5 Remove the entire `[diff.cpp23.containers]` section

[ Editor's note: Remove the entire [diff.cpp23.containers] section, as shown below. ]

1 Affected subclause: [span.overview]

**Change:** `span<const T>` is constructible from `initializer_list<T>`.

**Rationale:** Permit passing a braced initializer list to a function taking `span`.

**Effect on original feature:** Valid C++ 2023 code that relies on the lack of this constructor may refuse to compile, or change behavior in this revision of C++.

[Example 1:

```
void one(pair<int, int>);           // #1
void one(span<const int>);         // #2
void t1() { one({1, 2}); }         // ambiguous between #1 and #2; previously
    called #1

void two(span<const int, 2>);
void t2() { two({{1, 2}}); }      // ill-formed; previously well-formed

void *a[10];
int x = span<void* const>{a, 0}.size(); // x is 2; previously 0
any b[10];
int y = span<const any>{b, b + 10}.size(); // y is 2; previously 10
```

— end example]

## § 9 Appendix A: Original (pre-LEWG-review) proposed fix

This is the fix that the original version of this paper proposed. LEWG reviewed this on 2026-03-25 and polled to select a different fix.

### § 9.1 Ansatz: Constrain the initializer list constructor

Constrain `span(initializer_list<value_type>)` constructor so that `value_type` is not `bool`.

```
constexpr
explicit(extent != dynamic_extent)
span(initializer_list<value_type>)
requires(
    is_const_v<ElementType &&
    (! is_same_v<value_type, bool>) // ADD
);
```

That would have the advantage that everything else works fine still, but we can avoid the breakage of valid user code, so only `span<const bool>` is affected.

```
// Works in current C++26 draft;
// fails to compile with the above change
std::span<const bool> span_from_bool{true, false, true};

// Work-around: Enclose initializer_list in ()
std::span<const bool> span_from_bool2({true, false, true});
assert(span_from_bool2.size() == 3u);

// Work-around: Enclose initializer_list in {}
std::span<const bool> span_from_bool3{{true, false, true}};
assert(span_from_bool3.size() == 3u);
```

## § 9.2 Fix initializer list construction from actual bool values

The above change would break existing code, specifically by making the following fail to compile.

```
std::span<const bool> span_from_bool{true, false, true};
```

We can fix this by adding a new `initializer_list` constructor overload specifically for `bool` input values.

```
// NEW CONSTRUCTOR
template<typename InitListType>
constexpr
explicit(extent != dynamic_extent)
my_span(initializer_list<InitListType> il)
requires (is_const_v<ElementType>
    && (is_same_v<value_type, bool>)
    && (is_same_v<InitListType, bool>))
);
```

Here is an [implementation](#) (thanks to Giuseppe D'Angelo! who suggests that this constructor could be a C++29 feature, since it's an extension, as code would move from breaking to non-breaking).

## § 9.3 Fix initializer list construction from any non-pointer type convertible to bool?

It would be excellent to permit non-pointer types that are convertible to `bool`. For example, this would permit the common case of initializing `bool` values from 0 and 1 `int` literals.

```
std::span<const bool> span_from_bool{1, 0, 1};
```

However, we're not sure how to specify this. Merely replacing the `is_same_v<InitListType, bool>` constraint with `is_integral_v<InitListType>` does *not* work. Doing that results in `span<const bool>{0, 1, 0}` selecting the `initializer_list<value_type>` constructor, which fails with a hard error, because it



attempts to initialize the `const bool*` pointer with a `const int*` from the input `initializer_list<int>::const_iterator`.